

PreflightQC Dependency Research Report: FFmpeg / ffprobe

Commercial Distribution

Prepared August 2026. **This is not legal advice**; a qualified software-IP attorney should review the final architecture, notices, and EULA before commercial release.

TL;DR

- Ship an unmodified, official LGPL-2.1 shared build of ffprobe (and its libav DLLs/dylibs), invoke it as a separate child process, and bundle NO GPL or nonfree component* (no libx264, libx265, libxvid, libfdk-aac, libnpp, frei0r, libvidstab, etc.). This keeps PreflightQC closed-source with minimal, well-understood LGPL obligations.
- **ffprobe alone covers nearly all required metadata**; add MediaInfo (BSD-2-Clause, essentially obligation-free) as a complementary reader for the few fields ffprobe handles weakly — scan-type/field-order heuristics and clean Dolby Vision profile/HDR-format naming.
- **Pure inspection carries low patent risk and no encoder-license exposure**; the last significant US AVC/H.264 essential patent (US 7,826,532) expires 29 Nov 2027 while HEVC remains heavily licensed with patents extending well past 2030 — but because you never encode and only parse (decoding at most a frame or two for HDR side-data), practical exposure is minimal, though not formally zero per FFmpeg’s own patent FAQ.

Key Findings

1. FFmpeg’s default license is **LGPL v2.1-or-later**; GPL applies only if you explicitly enable GPL parts. LICENSE.md is verbatim: *“Most files in FFmpeg are under the GNU Lesser General Public License version 2.1 or later (LGPL v2.1+)... In combination the LGPL v2.1+ applies to FFmpeg. [github](#) Some optional parts... are licensed under the GNU General Public License version 2 or later (GPL v2+)... None of these parts are used by default, you have to explicitly pass --enable-gpl to configure to activate them. In this case, FFmpeg’s license changes to GPL v2+.”*
[github](#) [GitHub](#)
2. LGPL compliance for PreflightQC is achievable with dynamic linking + source availability + notices.
3. GPL is triggered by `--enable-gpl` components (e.g., libx264/libx265) or the GPL-only internal files.
4. `--enable-nonfree` produces a binary FFmpeg’s LICENSE.md says *“will cause the resulting binary to be unredistributable”* [GitHub](#) — an absolute no-ship.
5. MediaInfo/libmediainfo/ZenLib are BSD-2-Clause/zlib — the lowest-friction option, requiring only an attribution sentence.
6. ffprobe is sufficient for the metadata list; MediaInfo is complementary, not redundant, for a few fields.

Official Source Register

Source (URL)	What it establishes
	Primary licensing statement (LGPL 2.1+ default; GPL if GPL parts used; <i>“FFmpeg is not available under any other licensing terms, especially not proprietary/commercial ones, not even in exchange for payment”</i>); FFmpeg the 18-point

ffmpeg.org/legal.html	LGPL compliance checklist (dynamic linking, ship source, notices, EULA edits, DLL-naming rules); patent mini-FAQ (<i>"MPEG LA is vigilant and diligent about collecting for MPEG-related technologies"</i>). <code>ffmpeg</code>
github.com/FFmpeg/FFmpeg/blob/master/LICENSE.md (mirrors git.ffmpeg.org)	Per-file/per-library license map; enumerates GPL-v2 external libs (avisynth, frei0r, libcdio, libdavs2, librubberband, libvidstab, libx264, libx265, libxavs, libxavs2, libxvid), LGPL-v3 libs (gmp, libaribb24, liblensfun), GPL-v3 (smbclient), and nonfree (Fraunhofer FDK AAC; OpenSSL in incompatible combinations) → "unredistributable."
ffmpeg.org/download.html	Official links to recommended binary distributors (BtbN, gyan.dev/Windows, evermeet.cx/macOS).
github.com/BtbN/FFmpeg-Builds	Offers explicit <code>lgpl</code> , <code>lgpl-shared</code> , <code>gpl</code> , <code>gpl-shared</code> , <code>nonfree</code> variants for win64/linux/arm; the only mainstream distributor shipping LGPL <i>shared</i> Windows builds.
gyan.dev/ffmpeg/builds	Windows builds; <i>"All builds are 64-bit, static and licensed as GPLv3"</i> ; <code>CODEX FFmpeg</code> supplementary tools LGPLv3. <code>CODEX FFmpeg</code> → gyan's main builds are GPL, unsuitable for bundling.
evermeet.cx/ffmpeg	macOS static builds; observed configuration includes <code>--enable-gpl --enable-libx264 --enable-libx265</code> → GPL, unsuitable for bundling; Intel-only, runs on Apple Silicon via Rosetta.
gnu.org/licenses/old-licenses/lgpl-2.1.en.html	LGPL 2.1 §6 relink/object-file vs shared-library-mechanism obligations.
gnu.org/licenses/gpl-faq.html (#MereAggregation)	FSF position: pipes/sockets/command-line args = normally separate programs; mere aggregation permitted.
mediaarea.net/en/MediaInfo/License	MediaInfo BSD-2-Clause; binary-form attribution sentence requirement; ZenLib zlib license.
videolan.org/developers/x264.html, x264.org/licensing, x265.org, videolan.org/developers/x265.html	x264/x265 dual GPL-2-or-later/commercial; incorporating into a closed product requires a commercial license.
via-la.com (AVC/H.264), accessadvance.com (HEVC)	Patent pool status: AVC last US patent (US 7,826,532) expires 29 Nov 2027; HEVC Access Advance rates locked through 2030, 27,000+ patents.
ffmpeg.org/ffprobe.html, ffprobe-all.html; github.com/FFmpeg/FFmpeg/blob/master/fftools/ffprobe.c	ffprobe capability/field documentation; <i>"ffprobe gathers information from multimedia streams and prints it in human- and machine-readable fashion"</i> ; <code>debian</code> <code>-show_streams/-show_format/-show_frames/-show_data/side_data</code> behavior.

Licensing Analysis (19 Questions)

1. Default FFmpeg licence. LGPL v2.1-or-later. LICENSE.md: *"Most files in FFmpeg are under the GNU Lesser General Public License version 2.1 or later (LGPL v2.1+)... In combination the LGPL v2.1+ applies to FFmpeg."* `GitHub`
 GPL parts are off by default. (Established by primary source.)

2. LGPL obligations. From ffmpeg.org/legal.html's own checklist: build **without** `--enable-gpl / --enable-nonfree`; use dynamic linking (on Windows, link to DLLs); distribute the corresponding FFmpeg source (modified or not) that "corresponds exactly to the library binaries"; explain your configure line in a text file; host source on the same webserver as the binary; add the *"This software uses code of FFmpeg licensed under the LGPLv2.1 and its source can be downloaded here"* [ffmpeg](#) notice to your download page; put *"This software uses libraries from the FFmpeg project under the LGPLv2.1"* [ffmpeg](#) in your about box; mention FFmpeg/LGPLv2.1 in your EULA; if your EULA claims ownership of code, explicitly disclaim ownership of FFmpeg; remove any reverse-engineering prohibition from your EULA (and all translations); don't rename the DLLs to obfuscated names (suffixes/prefixes like `avcodec-MyProg.dll` are OK, `MyProgDec.dll` is not). (Primary source.)

3. What makes a build GPL. Passing `--enable-gpl`, or enabling any GPL-licensed external library (`libx264`, `libx265`, `libxvid`, `libxavs/libxavs2`, `libdav1d`, `frei0r`, `libcdio`, `librubberband`, `libvidstab`, `avisynth`) or the GPL-only internal files (e.g., `vf_delogo.c`, `vf_hqdn3d.c`, `vf_cropdetect.c`, `x86asm_flac_dsp_gpl.asm`, `idct_mmx.c`). `smbclient` forces GPL-v3. [github](#) (Primary source: *LICENSE.md*.)

4. `--enable-gpl`. Activates GPL-covered parts; *"In this case, FFmpeg's license changes to GPL v2+."* [GitHub](#) That copyleft would then reach PreflightQC if it links those libraries — incompatible with closed source. (Primary.)

5. `--enable-nonfree`. Enables license-incompatible libs (Fraunhofer FDK AAC; OpenSSL in incompatible combinations; CUDA SDK components). *LICENSE.md* verbatim: *"If you wish to enable these libraries, even in circumstances that their license may be incompatible, pass `--enable-nonfree` to configure. This will cause the resulting binary to be unredistributable."* [GitHub](#) Never ship such a build. (Primary.)

6. Components to avoid.

- **GPL-only (force `--enable-gpl`):** `libx264`, `libx265`, `libxvid`, `libxavs`, `libxavs2`, `libdav1d`, `frei0r`, `libcdio`, `librubberband`, `libvidstab`, `avisynth`.
- **GPLv3/LGPLv3 via `--enable-version3`:** `smbclient` (*"The smbclient library is under the GPL v3, to combine it with FFmpeg, the options `--enable-gpl` and `--enable-version3` have to be passed to configure to upgrade FFmpeg to the GPL v3"*), [github](#) [GitHub](#) `libaribb24/liblensfun/gmp` (LGPLv3).
- **Nonfree (force `--enable-nonfree` → *unredistributable*):** Fraunhofer FDK AAC (`libfdk-aac`), OpenSSL in incompatible combinations, and CUDA SDK components `libnpp` / `cuda-nvcc` (in configure's `HWACCEL_LIBRARY_NONFREE_LIST`). [32blog](#) (Primary: *LICENSE.md* + *configure*.)

7. libx264 implications. GPL-2.0-or-later from VideoLAN, with a separate commercial license available (x264.org: *"When you incorporate the x264 Software... into your application, you are automatically licensed under the open source license (unless you already have a Commercial License from x264)"*). [X264](#) Enabling it makes FFmpeg GPL and would obligate open-sourcing PreflightQC unless a commercial x264 license is bought. Encoder — not needed. **Avoid.**

8. libx265 implications. Same structure: GPL-2.0-or-later from MulticoreWare with an on-request commercial license, [x265](#) **plus separate HEVC patent obligations** (licensees "are also responsible for separately acquiring licenses for any HEVC related patents"). [VideoHelp](#) Encoder — not needed. **Avoid.**

9. Static vs dynamic linking under LGPL. LGPL 2.1 §6 gives two compliance routes: (a) *"Accompany the work with the complete... 'work that uses the Library', as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable"* [GNU](#) (the object-file/relink route); or (b) *"Use a suitable shared library mechanism... one that (1) uses at run time a copy of the library already present... and (2) will operate properly with a modified version of the library"* [GNU](#) if interface-compatible. For a closed-source app, **dynamic linking (route b) is dramatically easier** — you avoid shipping proprietary object files. Static linking is *possible* but forces the object-file/relink route (route a) and is not recommended. LGPL 3 §4 uses the same structure (option 4d0 relink vs 4d1 shared mechanism). [powys](#) **Use shared libraries.** (Primary: *LGPL text*.)

10. Separate process vs linking. `ffprobe` is a standalone executable. Running it as a child process (spawn/exec + read stdout) is the cleanest posture. The FSF GPL FAQ (#MereAggregation) states verbatim: “*pipes, sockets and command-line arguments are communication mechanisms normally used between two separate programs. So when they are used for communi [GNU](#) cation, the modules normally are separate programs. But if the semantics of the communication are intimate enough, exchanging complex internal data structures, that too could be a basis to consider the two parts as combined into a larger program.*” [GNU](#) So even a GPL `ffprobe` used as a subprocess (exchanging only command-line args and JSON output — not internal data structures) is, per the FSF’s stated position, normally a separate work — but the FAQ also concedes this is “*a legal question, which ultimately judges will decide.*” [GNU](#) **Regardless, the safest posture is both (i) an LGPL build AND (ii) a separate process**, so process isolation is defense-in-depth, not your sole compliance mechanism. (*FSF FAQ + established practice.*)

11. Redistribution requirements. Ship alongside binaries: the LGPL 2.1 license text; the corresponding FFmpeg source (or same-server link) matching the shipped binary version; a notice identifying FFmpeg and its LGPL license; your changes (if any) as a `git diff` plus your configure line; MediaInfo’s attribution sentence + BSD-2-Clause text; and the license texts of any bundled LGPL sub-libraries. (*Primary: [legal.html](#) checklist.*)

12. Source-code / written-offer obligations. Under LGPL 2.1 §4/§6 you must make the corresponding library source available. FFmpeg’s checklist recommends **hosting the source tarball on the same webserver as your binary** (the “equivalent access from the same place” approach). GPL §3’s alternatives (accompany-with-source, or a 3-year written offer) apply only if you ship GPL parts — which you should not. The same-server-download approach is simplest and is FFmpeg’s own recommendation. (*Primary.*)

13. Copyright/licence notices. Preserve all upstream copyright notices; include the full LGPL-2.1 text; put “*This software uses libraries from the FFmpeg project under the LGPLv2.1*” in the about box; add the download-page and EULA notices; credit the IJG “*in the documentation accompanying your program*” [GitHub](#) if the libjpeg-derived files (`jfdctfst.c`, `jfdctint_template.c`, `jrevdct.c`) are present; include MediaInfo’s “*This product uses MediaInfo library, Copyright (c) 2002-2026 MediaArea.net SARL*” [MediaArea](#) sentence. (*Primary.*)

14. Unmodified official binaries + mere aggregation. Using unmodified official LGPL shared binaries is strongly preferable: it makes “source that corresponds exactly to the binaries” trivial (point to the distributor’s published source + configure line) and avoids you becoming a “modifier.” Shipping `ffprobe.exe` as a standalone tool beside your app is closer to “mere aggregation” (separate programs on the same medium), which the FSF FAQ says “*has no effect on the other program.*” [Fsf](#) [iu](#) (*Primary + FSF FAQ.*)

15. Patent considerations for inspection only. The FFmpeg patent FAQ ([legal.html](#)) declines to assert FFmpeg is patent-free and warns that once you “*start trying to make money from patented technologies, the owners of the patents will come after their licensing fees.*” [ffmpeg](#) However, the essential-patent pools (Via LA for AVC, Access Advance for HEVC) license *encoding and decoding* of the coded video, not container/header parsing. PreflightQC’s `-show_streams / -show_format` operations read container/codecs-parameter metadata — the official docs describe `ffprobe` as one that “*gathers information from multimedia streams*” [debian](#) and “*probe[s] the url content*” [debian](#) — and do not decode compressed frames.

The important nuance (confirmed via the `ffprobe` docs and `fftools/ffprobe.c`): extracting *frame-attached HDR side data* (Mastering Display metadata, Content Light Level, Dolby Vision `AV_FRAME_DATA_DOLBY_METADATA`, HDR10+) requires `-show_frames` — in practice `-read_intervals "%+#1"` to read just the first frame [Code Calamity](#) — which engages the decoder. FFmpeg’s source describes the related `find_stream_info` step as reading “*and decode[ing] the streams to fill missing information with heuristics,*” [GitHub](#) and the `-show_log` option surfaces “*logging information from the decoder about each frame,*” [ffmpeg](#) confirming `-show_frames` engages the decoder. By contrast `-show_data` merely dumps raw packet/extradata bytes [ffmpeg](#) (no decode), and `-count_packets` demuxes only. So for HDR fields you technically perform a minimal decode of one or two frames. **Practically the exposure is very low** — you decode for inspection, not playback or output — **but it is not formally zero**, and AVC/HEVC decode patents remain in force (AVC US patents to 29 Nov 2027; HEVC well beyond 2030). (*FFmpeg FAQ + docs/source; the “no decode for headers” framing is FFmpeg’s “probe” language, with the explicit decode nuance flagged as interpretation.*)

16. Windows packaging. Use the **Btbn** `win64-1gpl-shared` build (the mainstream source of LGPL *shared* Windows binaries; gyan.dev’s main builds are GPLv3 and unsuitable, and gyan will only produce LGPL builds on request and less frequently). [VideoHelp](#) Layout: a `bin\` folder containing `ffprobe.exe` and the shared DLLs (`avcodec-*.dll` , `avformat-*.dll` , `avutil-*.dll` , `swresample-*.dll` , plus `swscale-*.dll` / `avfilter-*.dll` if present). **Keep DLL names unobfuscated** (checklist rule 16). Code-sign your own executables and installer; you may re-sign the third-party DLLs with your Authenticode certificate for a clean SmartScreen/UAC experience, but **do not modify them** (or if you do, you become a “modifier” with source-correspondence duties). Ship the LGPL text + same-server source link in the installer. (*Distributor docs + checklist.*)

17. macOS packaging. No mainstream distributor ships an LGPL *shared* macOS build off-the-shelf: evermeet.cx and gyan are GPL, and evermeet is Intel-only (its maintainer states he will not provide native Apple Silicon builds). [Evermeet](#) Options: **(a, recommended)** build your own LGPL shared FFmpeg for macOS as a **universal2 (arm64 + x86_64)** binary with `--enable-shared --disable-static` and **no** GPL/nonfree flags; **(b)** build LGPL *static* and accept the §6 object-file/relink obligation (more burdensome for closed source). Bundle dylibs in `YourApp.app/Contents/Frameworks` , reference via `@rpath / @loader_path` , code-sign each dylib and the `ffprobe` helper, enable the **Hardened Runtime**, and **notarize** the app. Prefer universal2 so you don’t depend on Rosetta (which Apple will eventually deprecate). [FFmpeg](#) (*Distributor evidence + Apple packaging norms; self-building is common practice.*)

18. Is ffprobe alone sufficient? Yes for essentially all listed fields via `-show_streams` , `-show_format` , `-show_frames` , and `side_data_list` in JSON. Limitations: field-order/scan-type detection is heuristic and often returns `field_order=unknown` for interlaced content (per FFmpeg users, ffprobe “*doesn’t determine the actual content field order*”; [VideoHelp](#) the `-vf idet` filter is a separate, non-authoritative heuristic). [VideoHelp](#) HDR/DV side data needs `-show_frames` .

19. MediaInfo + ffprobe — necessary or redundant? Complementary, not redundant. MediaInfo (BSD-2-Clause; switched from LGPL/GPL at v0.7.63) [Wikipedia](#) gives cleaner high-level naming for **HDR Format / Dolby Vision profile** (its “HDR Format” field distinguishes Dolby Vision vs HDR10/SMPTE ST 2086) [AVS Forum](#) and **Scan type / Scan order** derived from container atoms (e.g., QuickTime `fiel`), where ffprobe is weaker. ffprobe gives more granular per-stream/per-frame raw fields and machine-friendly JSON. Using both, with MediaInfo as the “gap filler,” is a defensible design — and MediaInfo’s licensing is so light (only an attribution sentence) it adds negligible obligation.

Allow / Deny Dependency Table

Component	Licence	Needed?	Commercial redistribution	Conditions	Decision
FFmpeg LGPL shared build	LGPL 2.1+	Yes	Allowed	Dynamic link, ship source+notices, no DLL obfuscation	ALLOW WITH CONDITIONS
FFmpeg GPL build	GPL 2+/3	No	Only if whole app GPL	Would force open-sourcing	DO NOT BUNDLE
ffprobe binary (from LGPL build)	LGPL 2.1+	Yes	Allowed	Run as separate process; ship notices/source	ALLOW WITH CONDITIONS
ffmpeg binary	LGPL/GPL per build	No (no encode in V1)	Depends	Not required for inspection	DO NOT BUNDLE (V1)
libavcodec / libavformat /					ALLOW

libavutil / libavfilter / libswscale / libswresample	LGPL 2.1+	Yes (as ffprobe deps)	Allowed	Shared/dynamic, notices	WITH CONDITIONS
libx264	GPL-2-or- later / commercial	No	Only under GPL or paid license	Encoder; forces GPL	DO NOT BUNDLE
libx265	GPL-2-or- later / commercial	No	Only under GPL or paid license	Encoder; +HEVC patents	DO NOT BUNDLE
libfdk-aac	Nonfree	No	Not redistributable	<code>--enable-nonfree</code>	DO NOT BUNDLE
libvidstab	GPL-2	No	Only under GPL	Filter; forces GPL	DO NOT BUNDLE
frei0r	GPL-2	No	Only under GPL	Forces GPL	DO NOT BUNDLE
libzvbi	GPL-2+	No	Only under GPL	Teletext; forces GPL	DO NOT BUNDLE
libaribb24	LGPL-3	No	Allowed under <code>--enable- version3</code>	Would upgrade FFmpeg to (L)GPLv3	DO NOT BUNDLE
libsmbclient	GPL-3	No	Only under GPLv3	Forces GPLv3 (<code>--enable-gpl -- enable-version3</code>)	DO NOT BUNDLE
libnpp / CUDA SDK (cuda- nvcc)	Nonfree	No	Not redistributable	HWACCEL_LIBRARY_NONFREE_LIST	DO NOT BUNDLE
MediaInfo CLI	BSD-2- Clause	Optional (recommended)	Allowed	Attribution sentence	ALLOW
libmediainfo	BSD-2- Clause	Optional	Allowed	Attribution sentence	ALLOW
ZenLib	zlib	Transitive (MediaInfo dep)	Allowed	Attribution	ALLOW

Metadata Capability Matrix

Field	ffprobe exposure	Limitation / MediaInfo gap-fill
Codec	<code>-show_streams codec_name / codec_long_name</code>	Complete.
Profile	<code>-show_streams profile</code>	Complete for common codecs (may show unknown for some, e.g., DV).
Level	<code>-show_streams level</code>	Complete (<code>-99</code> / N/A when absent).

Pixel format	<code>-show_streams pix_fmt</code>	Complete.
Resolution	<code>-show_streams width / height</code> (<code>coded_width/height</code>)	Complete.
SAR / DAR	<code>-show_streams sample_aspect_ratio /</code> <code>display_aspect_ratio</code>	Present when signaled; else must be computed from W×H.
FPS	<code>r_frame_rate / avg_frame_rate</code>	Complete.
Time base	<code>time_base (codec_time_base legacy)</code>	Complete.
Bitrate	<code>stream bit_rate / max_bit_rate ; format</code> <code>bit_rate</code>	N/A when container omits it; RenderIO MedialInfo may compute from size/duration.
Duration	<code>-show_format duration ; stream duration /</code> <code>duration_ts</code>	Complete.
Stream count	<code>-show_format nb_streams</code>	Complete.
Audio codec / sample rate / channels	<code>-show_streams audio: codec_name ,</code> <code>sample_rate , channels , channel_layout</code>	Complete.
Colour metadata	<code>color_range , color_space ,</code> <code>color_transfer , color_primaries</code>	At stream level when signaled; else <code>unknown</code> .
HDR mastering display / CLL	<code>-show_frames + side_data_list with -</code> <code>read_intervals "%+#1"</code>	Requires reading the first frame (minimal decode); MedialInfo shows an “HDR Format” summary.
HDR10+ / Dolby Vision	<code>-show_frames side_data</code> (<code>AV_FRAME_DATA_DYNAMIC_HDR_PLUS ,</code> <code>AV_FRAME_DATA_DOVI_METADATA</code>)	ffprobe exposes raw side data; MedialInfo gives cleaner DV profile / HDR-format naming.
Field order	<code>-show_streams field_order</code>	Often <code>unknown</code> ; heuristic only; MedialInfo “Scan type / Scan order” often more informative.
Container metadata / tags	<code>-show_format tags , stream tags</code>	Complete.

Recommended Architecture (V1)

Comparative evaluation:

- *MedialInfo only*: lowest licensing friction (BSD-2-Clause) but weaker granular per-stream/frame JSON and HDR side-data detail than ffprobe. **Insufficient alone** for a QC tool that needs raw stream parameters.
- *ffprobe only*: covers nearly everything with clean JSON, but weaker at scan-type/field-order and human-friendly Dolby Vision naming; carries LGPL obligations (manageable).
- *Dynamically linked FFmpeg libraries (in-process)*: maximal control but you become an LGPL “linker” (must guarantee relink/replaceability), and any crash in libav* takes down your app. Higher engineering + compliance burden than a subprocess.
- **MedialInfo + separate-process ffprobe: CHOSEN.** Best coverage, cleanest compliance, process isolation, offline,

easy Windows-first packaging.

Chosen V1 architecture: PreflightQC (closed-source) invokes **an unmodified BtbN** `win64-lgpl-shared` **ffprobe** as a **child process** (JSON output over stdout) and **bundles MedialInfo CLI (BSD-2-Clause)** as a complementary reader for scan-type/field-order and Dolby Vision/HDR-format naming. **No ffmpeg encoder binary, no GPL/nonfree components, dynamic linking only, fully offline.** On macOS, self-build an LGPL shared/universal2 ffprobe (no GPL/nonfree flags) and bundle dylibs under `Contents/Frameworks` with `@rpath`, Hardened Runtime enabled, notarized.

This maximizes metadata coverage while keeping the licensing surface to (i) LGPL-2.1 obligations discharged by shared-linking + source availability + notices, and (ii) a single BSD attribution sentence. It is Windows-first (BtbN ships ready-made LGPL shared Windows binaries), offline (no network calls), and requires no encoding.

Required Third-Party-Notices Contents

1. **FFmpeg:** the sentence *"This software uses libraries from the FFmpeg project under the LGPLv2.1"*; the full LGPL-2.1 text; a statement that PreflightQC does not own FFmpeg and where the copyright holders can be found; the exact configure line / build variant used (or a pointer to BtbN's published build recipe); and a link to the corresponding FFmpeg **source**, hosted on the **same server** as the PreflightQC download and matching the shipped binaries' version.
2. **Any bundled LGPL sub-library** enabled in the build: its notice + license text.
3. **IJG credit** if the libjpeg-derived files are present in the build.
4. **MedialInfo / MedialInfoLib:** *"This product uses MedialInfo library, Copyright (c) 2002–2026 MediaArea.net SARL,"* `MediaArea` plus the BSD-2-Clause text.
5. **ZenLib:** zlib-license attribution *"(c) MediaArea.net SARL, zlib license"*). `MediaArea`
6. **EULA edits:** remove any reverse-engineering prohibition (or carve out FFmpeg); disclaim ownership of FFmpeg; add the download-page notice with the source link; apply the same edits to all EULA translations.

Remaining Legal Uncertainty

- **"Corresponds exactly" for third-party binaries:** relying on BtbN's published source + recipe to satisfy LGPL §6 source-correspondence is standard practice, but you are trusting a third-party build; the safest posture is to **archive the exact source tarball + build config** that produced the binaries you ship.
- **Static-vs-shared edge cases**, and whether **re-signing** the DLLs with your certificate counts as "modification" (conservative view: signing is not modification of the code, but confirm with counsel).
- **The subprocess / "mere aggregation" line** is, in the FSF's own words, *"a legal question, which ultimately judges will decide"*; `GNU` process isolation is strong but not adjudicated for this exact scenario.
- **Minimal-decode for HDR side data** technically invokes a decoder; whether inspection-only decode implicates AVC/HEVC decode patents is untested and jurisdiction-dependent.
- **macOS LGPL shared build** must be self-produced; you own that build's correctness and compliance.
- **Patent-pool dynamics:** AVC and HEVC pool royalty structures are actively being restructured upward (Access Advance imposed a documented 25% new-licensee rate increase phased in during 2026); `GIGAZINE` these affect *encoders/decoders shipped for use*, not inspection tools, but confirm scope with counsel if any decode-for-inspection concern is material.

Final Gate: **APPROVED WITH CONDITIONS**

Conditions (all must hold):

1. Use only an **unmodified official LGPL (non-GPL, non-nonfree) shared build** of ffprobe + libav*; Windows via BtbN win64-lgpl-shared ; macOS via a self-built LGPL shared/universal2 binary.
2. **Invoke ffprobe as a separate process**; do not statically link or link libav* into the closed-source app.
3. **Bundle no GPL or nonfree component** (explicitly: no libx264, libx265, libxvid, libfdk-aac, libnpp/cuda-nvcc, frei0r, libvidstab, libzvbi, libsmbclient, libaribb24) and ship **no ffmpeg encoder** in V1.
4. Ship **LGPL-2.1 text + corresponding FFmpeg source (same-server link, version-matched) + notices**; keep DLL names unobfuscated; edit the EULA per the FFmpeg checklist (remove reverse-engineering ban, disclaim FFmpeg ownership).
5. Add MediaInfo only under BSD-2-Clause with its verbatim attribution sentence.
6. Treat all codec **encoding** as out of scope for V1; if HEVC/AVC encoding is ever added, obtain x264/x265 commercial licenses **and** the relevant codec patent-pool licenses first.
7. **Attorney review** of the final notices, EULA, and build manifest before release.

Reminder: This report synthesizes primary FFmpeg/GNU/MediaArea/VideoLAN/patent-pool sources with common-practice interpretation where flagged. It is not legal advice.